EPSR-test-ex47 alignmole: single water molecules

Program(s): alignmole | chained passes: 1 | verification: ALL IDENTICAL (1 pass)

What this case tests

Exercises the standalone program `alignmole`, which rotates every molecule of an `.ato` file [1] by Monte Carlo trial rotations, accepting only moves that do not increase the mean square distance of its atoms (all, or a listed subset) from a chosen Cartesian axis, e.g. to align molecules before building a lattice or a picture. Random numbers come from `ran1` [2,3].

Code exercised

Fortran (original, fortran_src/):

- `alignmole.f`: `alignmole` (Monte Carlo loop, downhill-only acceptance, step-size adaptation every `niter/10` trials), `displacement` (mean square distance from the axis)
- `molfit.f`: `rotatm` (random rotation of a molecule about its centre); `auxiliary_routines.f`: `ran1`
- `ato_read_and_save.f`: `minitc`, `moloutc` (reading / rewriting the `.ato`, random-number state included)

Rust (port, rust/epsr-core/src/):

- `tools/alignmole.rs`: `run`, `displacement`
- `tools/groupc_common.rs`: `rotatm`, `moloutc`; `rng.rs`: `Ran1::ran1`
- `ato.rs`: `State::minitc`

Physical system

One water molecule (O-H 0.976 Å, from the EPSR-test `WaterMolecule.ato`) in a cubic 14.42 Å box: `waterz.ato` with the molecule in the xy plane, aligned to the z axis using all atoms; `waterx.ato` with the molecule in the yz plane, aligned to the x axis using atoms 1 2 3 (atom-list branch). Rotational step size `stepr` = 0.05 from the `.ato` header, 10 iterations each.

Inputs

Both files built from `WaterMolecule.ato` by a helper script (a one-off script that was not kept; the generated file is the input) with the random generator re-seeded by `idum` = `-seed` (seeds 61 and 329, found by search). Commands `alignmole waterz.ato z 10 all` and `alignmole waterx x 10 1 2 3`.

How the accuracy was verified

`commands.txt` (2 program calls, run once) was run with the Fortran reference build (`rust/tools/build_fortran_reference.sh`: the original sources, gfortran -O2 -fno-tree-vectorize, single thread) and with the Rust `epsr-rs` on identical copies of the inputs, and every file written was compared byte for byte (`rust/tools/run_ex_test.sh`, `compare_outputs.py`; no masking except where noted below). The Fortran output is stored in `fortran_reference.tar.gz`. Result of the final regression run of the complete port: every file identical in 1 pass.

Files compared: `waterz.ato`, `waterx.ato` (rotated coordinates, advanced random-number state, step size restored to 0.05). Starting at the maximum of the displacement and with these seeds, all 10 trial rotations decrease the displacement (the molecule leaves its plane), and the step size is adapted at iteration 10. Compared against the original Fortran binary.

Notes

Limitation of the Fortran: a rejected trial rotation restores the molecule from the work arrays `xr, yr, zr`, which are allocated but never assigned, so uninitialised heap memory is copied into the coordinates; such runs cannot be reproduced (the port uses zero-filled arrays and prints a warning), and this case is designed so that no rotation is rejected. `niter` < `10` makes the Fortran divide by zero (`naccum` = `niter/10`); the port reports an input error.

References

[1] A. K. Soper, "Empirical Potential Structure Refinement - EPSRshell: a user's guide", Technical Report RAL-TR-2011-012, STFC Rutherford Appleton Laboratory (2011).
[2] S. K. Park and K. W. Miller, "Random number generators: good ones are hard to find", Commun. ACM 31, 1192-1201 (1988).
[3] W. H. Press, S. A. Teukolsky, W. T. Vetterling and B. P. Flannery, Numerical Recipes in Fortran 77, 2nd ed., Cambridge University Press (1992).
[4] GNU Fortran (gfortran) 15.2, GCC, https://gcc.gnu.org/fortran/ - reference build with -O2 -fno-tree-vectorize.